

5.1

A service-oriented architecture IS a component-based architecture, where the components are implemented by thinking about them as services, self-contained programs that performs a task for clients.

5.2

Due to the simplicity of this app, and seemingly no need for it to connect to the internet or databases of any kind, a Monolithic architecture seems to make the most sense. There's no need for a client-server relationship because nothing is stored outside of the local application. Due to the procedural, repetitive nature of the tic-tac-toe game flow, an event-driven or rule-based architecture might make sense too.

5.4

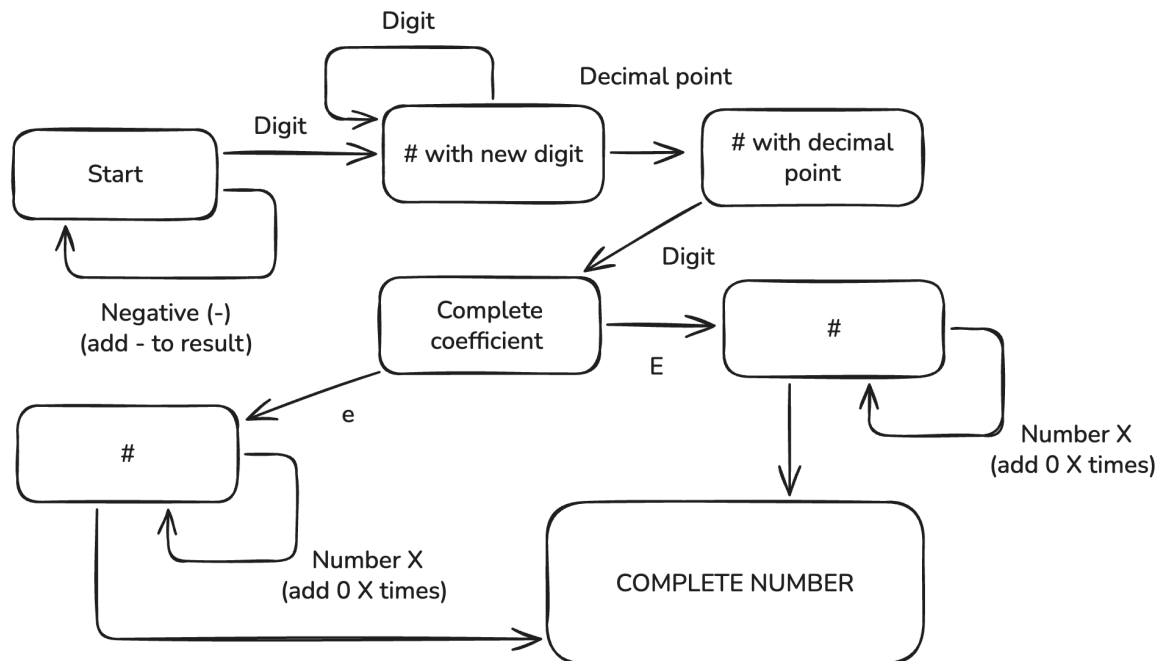
For a chess game that lets you play over the Internet with another player, I'd definitely want to go for a two-tier(or possible even more tiers) architecture. We have multiple user interfaces connected over a network, that may need to be associated with a database that can track scores and past games. this architecture will most likely need to be event-driven, reacting to state changes on the board, and evaluating scores and updating the UI accordingly.

5.6

Unless the developer intends to enable cloud saving on ClassyDraw, I would assume storage would happen locally, in a unique file type designed specifically to store ClassyDraw art pieces. We can design this local database in a way that enables easy maintenance, sorting

drawings by how long ago they were last modified, and creating a backup folder that refreshes every month or so, so no dead files linger around.

5.8



6.1

All the shapes in the ClassyDraw application share properties such as "Drag", "PointIsAt", and some styling operations such as "Rotation" and "FillColor", which should be defined in the superclass "Drawable". Text is more unique of a class and will contain stuff like "Content" and "Style". What they don't share are their individual implementations of PointIsAt, which determines their shape to begin with. Nonshared properties should be implemented in the subclasses themselves.

6.2

